

CLI File Encryptor/Decryptor (Project Report)

Introduction

Nowadays, everyone—from big companies to individual users—has to keep files like documents, photos, or sensor logs safe on all kinds of devices and services. While tools like OpenSSL are well known for encrypting data, they can be hard to use. You need to remember tricky command-line options, set up salts and IVs yourself, and run extra commands just to check that nothing was tampered with. All these steps can lead to mistakes, skipped checks, and, in the worst case, unprotected data.

This project offers a single, easy-to-use Python script called **cli_file_encryptor.py** that bundles the most reliable encryption techniques into one tool. It automatically uses AES-CBC to scramble your files, HMAC-SHA256 to verify they haven't been changed, and PBKDF2 to strengthen your password. As it runs, the script gives clear updates on how long each step takes. It works on both normal computers and small IoT devices, so anyone—whether you're a beginner or a pro—can protect files with top-grade security.

Motivation and Technical Approach

Everyone knows that even the best encryption can fail if it's set up wrong. If you accidentally reuse salts or IVs, or skip integrity checks, attackers can tweak your encrypted data or replay old messages without you noticing. Our script fixes this by doing everything for you: for each file, it makes a brand-new 16-byte salt and IV. It uses PBKDF2-HMAC-SHA256 with 200,000 rounds to turn your password into two separate keys—one for encrypting, one for checking integrity—so it's much harder for someone to guess weak passwords. After padding the file with PKCS7, it encrypts with AES-CBC (you pick 128- or 256-bit), then computes an HMAC over the salt, IV, and ciphertext. If anything's changed, the script stops immediately before trying to decrypt.

Instead of making you run lots of commands, the tool has just two straightforward options: **encrypt** and **decrypt**, each with sensible defaults and easy flags for setting your password and choosing key size. Behind the scenes, functions like `derive_keys()`, `encrypt_file()`, and `decrypt_file()` handle all the crypto work, file reading/writing, and error checks. When it finishes, the script prints how fast it ran—so you can see right away how your computer and chosen settings affect performance. That's something you don't get with most standard tools.

Implementation Highlights

Encrypted files follow a compact binary structure that begins with a six-byte magic header (CLIENC), followed by a version byte and a one-byte key-length indicator. The 16-byte salt and 16-byte IV appear next, trailed by the ciphertext and a 32-byte HMAC tag. This layout makes parsing straightforward and tamper evident.

Key components of the code include:

- **Key Derivation:** PBKDF2 stretches the user's password into a concatenated encryption key and HMAC key, ensuring per-file randomness via the salt.
- **Encryption/Decryption:** AES-CBC provides confidentiality, while PKCS7 padding ensures block alignment. On decryption, the script verifies the HMAC and aborts immediately if authentication fails, protecting against both tampering and password errors.

- **User Experience:** The argparse-driven interface prompts for a password if none is provided as a flag, guiding the user through each step and avoiding surprises.

The entire tool depends only on Python 3.6+ and the PyCryptodome library, making installation as simple as `pip install pycryptodome`.

Performance Benchmarking

We wanted to see how fast our tool really is, so we ran it side-by-side with OpenSSL’s AES-CBC on random files of 1 MB, 10 MB, and 100 MB. A simple shell script handled each test: it timed the runs and worked out the speed in MB/s. The results fell into two clear speed ranges:

File Size	Python Tool (MB/s)	OpenSSL (MB/s)	Difference
1 MB	2.14	1.85	+15.6%
10 MB	17.56	77.78	-77.4%
100 MB	56.95	306.34	-81.4%

- **Small Files (1 MB):** PBKDF2 key stretching dominates the runtime. Our script, which uses PyCryptodome’s C-accelerated PBKDF2, achieved 2.14 MB/s compared to OpenSSL’s 1.85 MB/s, a 15.6 % advantage.
- **Large Files (10 MB and 100 MB):** Bulk encryption speed becomes the critical factor. OpenSSL’s hand-optimized routines reached roughly 300 MB/s, whereas our Python solution sustained about 57 MB/s—demonstrating that while native binaries excel at high throughput, our tool’s speed remains quite serviceable for most use cases.

Moreover, our script’s built-in HMAC computation adds an extra pass over the data, a cost that is absent in a bare `openssl enc` invocation but essential for integrity.

Real-World Validation

We didn’t stop at speed tests—we also tried real files to make sure everything works correctly. When we encrypted and then decrypted a plain-text document, every character came back exactly the same. Random binary “blobs” round-tripped without a hitch, and JPEG photos from our Testing folder returned as perfect, byte-for-byte copies. If even a single byte of the encrypted data is changed, the script immediately fails the HMAC check and refuses to decrypt, and if you enter the wrong password it stops with a clear error message. These real-world trials show that the tool is rock-solid under any conditions.

Contributions and Broader Impact

By integrating encryption, authentication, and benchmarking into one script, `cli_file_encryptor.py` addresses a clear gap in existing tooling. It enables:

- **Edge-to-Cloud Security:** IoT devices can encrypt logs and sensor data locally before transmission, ensuring confidentiality even over untrusted networks.
- **Simplified Workflows:** Administrators and non-expert users gain a straightforward command that enforces best practices by default, reducing the chance of critical misconfiguration.

- **Adaptable Security Levels:** The choice between AES-128 for speed and AES-256 for maximum strength allows deployments to balance performance and risk.

These features make the tool particularly well suited for mobile and IoT scenarios, where limited interfaces and constrained resources demand both simplicity and security. The inclusion of performance metrics empowers developers to make informed decisions when tailoring encryption settings to hardware capabilities.

Challenges and Future Directions

Choosing how many PBKDF2 rounds to use was a balancing act: we wanted strong protection against password attacks but still fast enough that users don't get stuck waiting. We settled on 200,000 rounds as a good middle ground for small files. In the future, switching to an AEAD cipher like AES-GCM would let us encrypt and authenticate in one step, making things both faster and simpler. We're also looking at adding multithreading or C-based speedups to handle big files more quickly, and maybe even building a graphical or mobile app so it's easier for everyone to use. Finally, tying into hardware security modules—like TPM chips or ARM TrustZone—could keep keys safe in a tamper-proof environment.

Conclusion

This project shows that a straightforward Python script can give you top-level file encryption and integrity checks without the usual complexity. While specialized, lower-level libraries might still be faster for huge amounts of data, **cli_file_encryptor.py** shines with its easy setup, clear feedback, and secure defaults. It works equally well on desktop computers or small battery-powered IoT devices. By making strong, correct encryption simple to use, this tool helps make sure no file—big or small—gets left unprotected because of a configuration mistake or oversight.

How to Run

Getting started with **cli_file_encryptor.py** requires only a few steps:

1. **Install Dependencies**

Ensure you have Python 3.6 or later. Then install PyCryptodome:

Code

```
pip install pycryptodome
```

2. **Place the Script**

Save `cli_file_encryptor.py` and `benchmark.sh` in your project folder.

3. **Prepare Benchmark Files** (optional)

If you want to compare performance, create random-data files:

Code:

```
dd if=/dev/urandom of=Files_for_benchmark/1MB.bin bs=1m count=1

dd if=/dev/urandom of=Files_for_benchmark/10MB.bin bs=1m
count=10

dd if=/dev/urandom of=Files_for_benchmark/100MB.bin bs=1m
count=100

chmod +x benchmark.sh
```

4. Encrypt and Decrypt

To encrypt a file:

Code:

```
python3 cli_file_encryptor.py encrypt \
-i path/to/input.file \
-o path/to/output.enc \
--key-size 256
```

You will be prompted for a password if you don't supply -p "YourPass".

To decrypt:

```
python3 cli_file_encryptor.py decrypt \
-i path/to/output.enc \
-o path/to/recovered.file
```

5. Run Benchmarks

From the project folder, simply execute:

```
./benchmark.sh
```

This will encrypt each blob in Files_for_benchmark/ with both your script and OpenSSL, printing side-by-side throughput and percentage differences.

6. Test on Real Files

You can also round-trip any file—images, PDFs, or archives—by pointing -i and -o at your chosen paths. For example:

```
python3 cli_file_encryptor.py encrypt \
-i Testing/photo.jpg \
-o Testing/photo.enc \
-p MySecret
```

```
python3 cli_file_encryptor.py decrypt \  
-i Testing/photo.enc \  
-o Testing/photo_recovered.jpg \  
-p MySecret
```

These steps will get you up and running, whether you're encrypting sensitive documents or validating performance on large datasets.